Introduction à JSX (JavaScript XML)

Objectifs

1. Définir ce qu'est JSX et comprendre son rôle dans l'écosystème ReactJS.
2. Expliquer comment le code JSX est transformé en appels de fonctions JavaScript standard (transpilation).
3. Maîtriser les règles syntaxiques fondamentales pour écrire du JSX propre et valide.
4. Injecter des expressions JavaScript dynamiques à l'intérieur des balises JSX.
5. Gérer les attributs, les styles en ligne et les classes CSS selon les conventions React.

1. Qu'est-ce que JSX ?

1.1 Définition et Historique

JSX (pour *JavaScript XML*) est une extension syntaxique du langage JavaScript créée par l'équipe de Facebook lors de la conception de React.

Contrairement aux idées reçues ou aux architectures traditionnelles qui prônent une séparation stricte des technologies (fichiers `.html` d'un côté, fichiers `.js` de l'autre), React fait le pari inverse : **la logique de rendu et le balisage visuel sont intimement liés au sein d'un même composant**.

```jsx
// Exemple d'un élément JSX simple stocké dans une constante
const element = <h1>Bonjour, technicien opérateur</h1>;
```

Ce code n'est ni une chaîne de caractères (`string`), ni du HTML pur. C'est une extension de syntaxe qui permet d'écrire des structures arborescentes directement dans du code JavaScript.

1.2 Le Rôle du Transpileur (Babel / SWC / esbuild)

Les navigateurs web ne comprennent pas nativement la syntaxe JSX. Si vous envoyez un fichier contenant des balises HTML au sein de code JavaScript directement à un navigateur, celui-ci déclenchera une erreur de syntaxe.

C'est ici qu'intervient le **transpileur** (intégré dans l'outil de build comme Vite ou Webpack). Avant que le code ne soit exécuté par le navigateur, le compilateur transforme chaque bloc

JSX en appels de fonctions JavaScript standard, plus précisément en appels à la fonction `React.createElement` (ou aux fonctions de transformation automatique du compilateur).

Code source écrit par le développeur (JSX) :

```jsx
const heading = <h1 className="title">Supervision Active</h1>;
```

Code produit après transpilation (JavaScript pur) :

```javascript
const heading = React.createElement(
  'h1',
  { className: 'title' },
  'Supervision Active'
);
```

Chaque élément JSX est donc converti en un objet JavaScript léger (un nœud du DOM Virtuel).

2. Règles Syntaxiques Fondamentales de JSX

Bien que JSX ressemble fortement au HTML, il obéit à des règles de syntaxe plus strictes dictées par la syntaxe JavaScript.

2.1 Un Élément Racine Unique

Un composant ou une expression JSX doit **toujours retourner un et un seul élément racine**. Si vous souhaitez retourner plusieurs éléments frères sans ajouter de balise HTML inutile dans le DOM (comme une `div` superflue), vous devez utiliser un **Fragment React** (`<> ... </>`).

```jsx
// INTERDIT : Plusieurs éléments frères sans conteneur racine
export default function BrokenHeader() {
  return (
    <h1>Titre</h1>
    <p>Sous-titre</p>
  );
}

// VALIDE : Utilisation d'un conteneur global
```

```
export default function ValidHeaderDiv() {
  return (
    <div>
      <h1>Titre</h1>
      <p>Sous-titre</p>
    </div>
  );
}

// VALIDE : Utilisation d'un Fragment React (aucun nœud créé dans le
DOM réel)
export default function ValidHeaderFragment() {
  return (
    <>
      <h1>Titre</h1>
      <p>Sous-titre</p>
    </>
  );
}
```

2.2 Fermeture Systématique de Toutes les Balises

En HTML, certaines balises dites "orphelines" ou auto-fermantes peuvent s'écrire sans balise de fermeture (par exemple `<br>`, `<img>`, `<input>`). **En JSX, toutes les balises doivent impérativement être fermées.** Les balises sans enfants doivent obligatoirement utiliser la syntaxe auto-fermante avec un slash final (`/>`).

```jsx
// INTERDIT en JSX
<br>
<img src="logo.png">
<input type="text">

// VALIDE en JSX
<br />
<img src="logo.png" alt="Logo de l'entreprise" />
<input type="text" />
```

2.3 Conventions de Nommage et Attributs HTML

Puisque JSX est converti en JavaScript et que certains mots-clés du HTML entrent en conflit direct avec les mots réservés du langage JavaScript (comme `class` ou `for`), les attributs HTML subissent une modification de nommage :

- **`class`** devient **`className`** en JSX.
- **`for`** (utilisé dans les labels de formulaires) devient **`htmlFor`**.
- Les attributs composés s'écrivent en **camelCase** : `tabindex` devient `tabIndex`, `onclick` (en minuscules en HTML) devient `onClick`, `maxlength` devient `maxLength`.

```jsx
// Exemple d'attributs convertis en conventions JSX
export default function StationForm() {
  return (
    <div className="form-container">
      <label htmlFor="station-name">Nom de la station :</label>
      <input id="station-name" type="text" tabIndex={1} />
    </div>
  );
}
```

3. Injection d'Expressions JavaScript dans le JSX

La grande puissance de JSX réside dans sa capacité à basculer instantanément du mode balisage au mode JavaScript en utilisant des **accolades `{}`**.

3.1 Insertion de Variables et d'Expressions

Tout ce qui se trouve entre accolades `{}` est évalué comme du code JavaScript standard. Vous pouvez y insérer des variables, des chaînes de caractères, des nombres, ou appeler des fonctions.

```jsx
export default function StationInfo() {
  const stationName = "Toulouse-Blagnac";
  const altitude = 152;
  const isOnline = true;

  return (
    <div className="card">
```

```
      <h2>Station : {stationName.toUpperCase()}</h2>
      <p>Altitude : {altitude + " mètres"}</p>
      <p>Statut technique : {isOnline ? "En service" : "Hors
ligne"}</p>
    </div>
  );
}
```

3.2 Ce qui ne peut PAS être affiché directement entre accolades

Certains types de données JavaScript ne peuvent pas être affichés directement par React dans le flux JSX :

- **Les objets JavaScript bruts** : `{ station }` provoquera une erreur critique d'exécution de React (*Objects are not valid as a React child*). Vous devez cibler une propriété précise (ex. `{ station.name }`).
- **Les booléens (`true` / `false`) et `null` / `undefined`** : React les ignore totalement lors du rendu. C'est une excellente propriété utilisée pour le rendu conditionnel, mais elle signifie qu'un booléen brut ne s'affichera pas sous forme de texte "true" ou "false".

4. Rendu Conditionnel et Listes en JSX

4.1 Le Rendu Conditionnel

Puisque JSX intègre du JavaScript pur, vous n'avez pas besoin de structures de contrôle propriétaires (comme des directives `v-if` ou `ng-if`). Vous utilisez les mécanismes natifs de JavaScript :

1. **L'opérateur ternaire (`condition ? vrai : faux`)** : idéal pour alterner entre deux affichages.
2. **L'opérateur logique ET (`condition && expression`)** : idéal pour afficher un bloc uniquement si la condition est vraie.

```
```jsx
export default function AlertBanner({ level, message }) {
  return (
    <div className="banner">
      {/* Affichage conditionnel d'un badge critique */}
      {level === 'critical' && <span
className="badge-alert">URGENT</span>}

      {/* Affichage ternaire du message */}
```
```

```
      <p>
        {message ? message : "Aucun incident signalé sur la ligne."}
      </p>
    </div>
  );
}
```

4.2 Le Rendu de Listes avec `.map()`

Pour afficher une collection de données (un tableau d'objets), on utilise la méthode fonctionnelle `.map()` de JavaScript combinée à du JSX. Chaque élément généré itérativement doit obligatoirement posséder un attribut spécial nommé **`key`** (généralement un identifiant unique stable, comme un ID technique).

```jsx
export default function StationsList() {
  const stations = [
    { id: 1, name: 'Toulouse-Blagnac', status: 'online' },
    { id: 2, name: 'Pic du Midi', status: 'offline' },
    { id: 3, name: 'Marseille-Marignane', status: 'online' }
  ];

  return (
    <ul>
      {stations.map(station => (
        <li key={station.id}>
          {station.name} - <strong>{station.status}</strong>
        </li>
      ))}
    </ul>
  );
}
```

5. Gestion des Styles en JSX

React propose deux manières principales d'appliquer des styles visuels aux éléments JSX :

5.1 Les Styles en Ligne (Inline Styles)

En HTML, l'attribut `style` prend une chaîne de caractères (ex. `style="color: red; margin-top: 10px;"`). En JSX, l'attribut `style` attend un **objet JavaScript** dont les propriétés sont écrites en **camelCase** (ex. `marginTop` au lieu de `margin-top`).

```jsx
export default function StyledPanel() {
  const boxStyles = {
    backgroundColor: '#f8fafc',
    border: '1px solid #cbd5e1',
    padding: '1rem',
    borderRadius: '6px'
  };

  return (
    <div style={boxStyles}>
      <h3 style={{ color: '#2563eb', fontSize: '1.25rem' }}>Panneau Sécurisé</h3>
      <p>Paramètres de chiffrement actifs.</p>
    </div>
  );
}
```

5.2 Les Feuilles de Style Externes et Classes CSS

Pour les projets professionnels de grande envergure, la méthode privilégiée reste l'utilisation de feuilles de style CSS traditionnelles importées au niveau des fichiers ou des composants, combinée à l'attribut `className`.

```jsx
import './StationCard.css';

export default function StationCard({ name, status }) {
  return (
    <article className={`station-card status-${status}`}>
      <h3>{name}</h3>
    </article>
  );
}
```

6. Synthèse

1. **Nature de JSX :** Une extension syntaxique combinant balisage et logique JavaScript au sein d'un même fichier, convertie par le transpileur en appels de fonctions standards.
2. **Règles Strictes :** Un seul élément racine obligatoire, fermeture systématique de toutes les balises, et remplacement de `class` par `className`.
3. **Puissance des Accolades `{}` :** Permettent d'injecter des expressions JavaScript dynamiques, de manipuler des conditions et de parcourir des listes avec `.map()`.
4. **Cohérence Visuelle :** Utilisation d'objets camelCase pour les styles en ligne et de classes CSS traditionnelles pour la mise en forme globale.